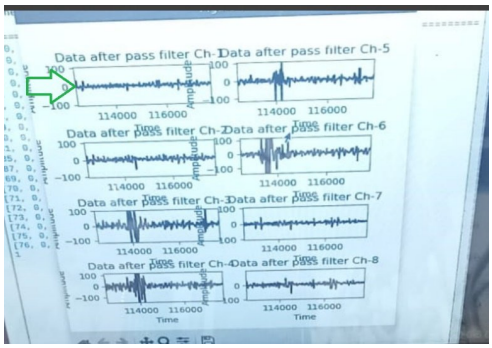
DATA INTERFACE & SOFTWARE GUIDE · SKU CQA240229P1

SPI Protocol, Registers & SDKs

Everything needed to read PiEEG-8 from your own code: SPI settings, the 27-byte frame, conversion to microvolts, default ADS1299 registers, PiEEG Server, the Python SDK, BrainFlow and the open datasets.



SPI0	27 B	250/s	0.268	ID 56
CE0 · mode 1	Frame per sample	Frames per second	µV per LSB (gain 1)	BrainFlow board ID

01 SOFTWARE OPTIONS

Option	Best for	Language	Start
PiEEG Server	Dashboard, recording, WebSocket, LSL, OSC, webhooks	Any (WebSocket JSON)	`pieeg-server --device pieeg8`
Python SDK	Learning the ADS1299, raw data, custom registers	Python	`1.8_ch_Pi5.py`
BrainFlow	Existing BrainFlow pipelines	Python, C++, Java, C#...	`BoardIds.PIEEG_BOARD`

02 SPI INTERFACE

Device	`/dev/spidev0.0`
Mode	1 (CPOL 0, CPHA 1)
Bit order	MSB first
Word	8 bits
Clock	600 kHz in SDK scripts; PiEEG Server 4 MHz (Pi 4), 2 MHz (Pi 5)
DRDY	GPIO 26, falling edge = new sample

Pi 4 DRDY chip	`/dev/gpiochip0` or RPi.GPIO pin 37
Pi 5 DRDY chip	`/dev/gpiochip4` (RP1); check with gpiodetect

03 DATA FRAME

byte 1	4	7					25	27
STATUS	CH1	CH2	CH3	CH4	CH5	CH6	CH7	CH8
0xC0 0x00 0x08	3 bytes	3 bytes	3 bytes	3 bytes	3 bytes	3 bytes	3 bytes	3 bytes

Figure 1 — Read 27 bytes each time DRDY goes low. Each channel is 24-bit big-endian two's complement.

µV = raw × 4.5 / 16 777 215 × 10^6

One count ≈ **0.268 µV** at gain 1. This is the conversion used by the SDK and PiEEG Server.

```python
raw = (b[0] << 16) | (b[1] << 8) | b[2]
if raw & 0x800000:          # negative value
    raw -= 16777214
uV = raw * 4.5 / 16777215 * 1_000_000
```


For research and education only. Not a medical device. Specifications subject to change.

04 ADS1299 COMMANDS AND DEFAULT REGISTERS

Command	Code
WAKEUP	0x02
RESET	0x06
START	0x08
STOP	0x0A
RDATAC	0x10
SDATAC	0x11

Addr	Register	Value	Meaning
0x01	CONFIG1	0x96	250 SPS
0x02	CONFIG2	0xD4	Test-signal settings
0x03	CONFIG3	0xFF	Internal reference and bias buffers on
0x04	LOFF	0x00	Lead-off off
0x05–0x0C	CH1SET–CH8SET	0x00	Normal input, gain 1
0x14	GPIO	0x80	
0x15	MISC1	0x20	SRB1 on: shared reference
0x17	CONFIG4	0x00	Continuous conversion

Write a register with `[0x40 | addr, 0x00, value]`, read with `[0x20 | addr, 0x00, 0x00]`. Start-up: WAKEUP, STOP, RESET, SDATAC, write registers, RDATAC, START.

CAUTION Keep 250 SPS for the standard tools.

Changing CONFIG1–3 changes the sample rate or reference. PiEEG Server and BrainFlow expect 250 SPS.

05 PIEEG SERVER

SHELL

```
pieeg-server --device pieeg8                    # stream + dashboard
pieeg-server --device pieeg8 --filter           # 1-40 Hz band-pass
pieeg-server --device pieeg8 --record session.csv # record while streaming
pieeg-server --device pieeg8 --lsl              # Lab Streaming Layer outlet
pieeg-server --device pieeg8 --osc              # VRChat OSC bridge
pieeg-server record session.csv --device pieeg8 --duration 300
pieeg-server monitor --device pieeg8            # live view in the terminal
pieeg-server doctor                             # check SPI, GPIO and settings
```

Address	What it is
http://raspberrypi.local:1617	Browser dashboard
ws://raspberrypi.local:1616	WebSocket stream: JSON frames with 8 channels in µV

READ THE STREAM FROM PYTHON

```
import asyncio, json, websockets

async def main():
    async with websockets.connect("ws://raspberrypi.local:1616") as ws:
        async for msg in ws:
            f = json.loads(msg)             # {"t": ..., "n": ..., "channels": [8 values in uV]}
            print(f["n"], f["channels"])

asyncio.run(main())
```

06 PYTHON SDK SCRIPTS

Script	Pi	What it does
1.8_ch_Pi5.py	Pi 5	Live 8-channel graph, band-pass (python3-libgpiod)
1.8_ch_Pi5_gpiod_1_5_4.py	Pi 5	Same, for gpiod==1.5.4
1.8_ch_Pi4.py	Pi 4	Live 8-channel graph, band-pass
Raw_data_1Ch_Pi5.py	Pi 5	Raw view of one channel
save_data_short_Pi5.py	Pi 5	Save raw µV to Excel

For research and education only. Not a medical device. Specifications subject to change.

Script	Pi	What it does
save_data_short_Pi4.py	Pi 4	Save raw µV to Excel

INSTALL AND RUN

```
sudo apt install -y python3-venv python3-dev
python3 -m venv --system-site-packages ~/pieeg-env && source ~/pieeg-env/bin/activate
pip install spidev matplotlib scipy pandas openpyxl
sudo apt install -y python3-libgpiod        # Pi 5 (or: pip install gpiod==1.5.4)
pip install RPi.GPIO                        # Pi 4
git clone https://github.com/pieeg-club/PiEEG-8.git && cd PiEEG-8/1.GUI
python 1.8_ch_Pi5.py                        # or 1.8_ch_Pi4.py
```

07 BRAINFLOW

`BoardIds.PIEEG_BOARD` (ID **56**): 8 EEG channels at 250 Hz named Fp1, Fp2, C3, C4, P7, P8, O1, O2.

CAUTION Build BrainFlow on the Pi.

SPI support is not in the pip wheels. Build from source with `python tools/build.py --build-periphery` (CMake -DBUILD_PERIPHERY=ON).

PYTHON

```
import time
from brainflow.board_shim import BoardShim, BrainFlowInputParams, BoardIds
params = BrainFlowInputParams(); params.serial_port = "/dev/spidev0.0"
board = BoardShim(BoardIds.PIEEG_BOARD, params)
board.prepare_session(); board.start_stream()
time.sleep(10); data = board.get_board_data()     # (8 EEG rows, ~2500 samples)
board.stop_stream(); board.release_session()
```

08 OPEN DATASETS

Dataset	Content
Datasets/dataset.xlsx	Raw EEG, dry Ag/AgCl without gel, 10-20 positions, 250 SPS, 8 channels, 77 s
1.GUI/dataset/	Raw EEG with chewing and blinking sequences, 250 SPS
2.Software/Dataset/eeg_data_3.txt	Raw 27-byte frames from the PiEEG app
ML_application/	Alpha-power feature extraction and binary classifier example

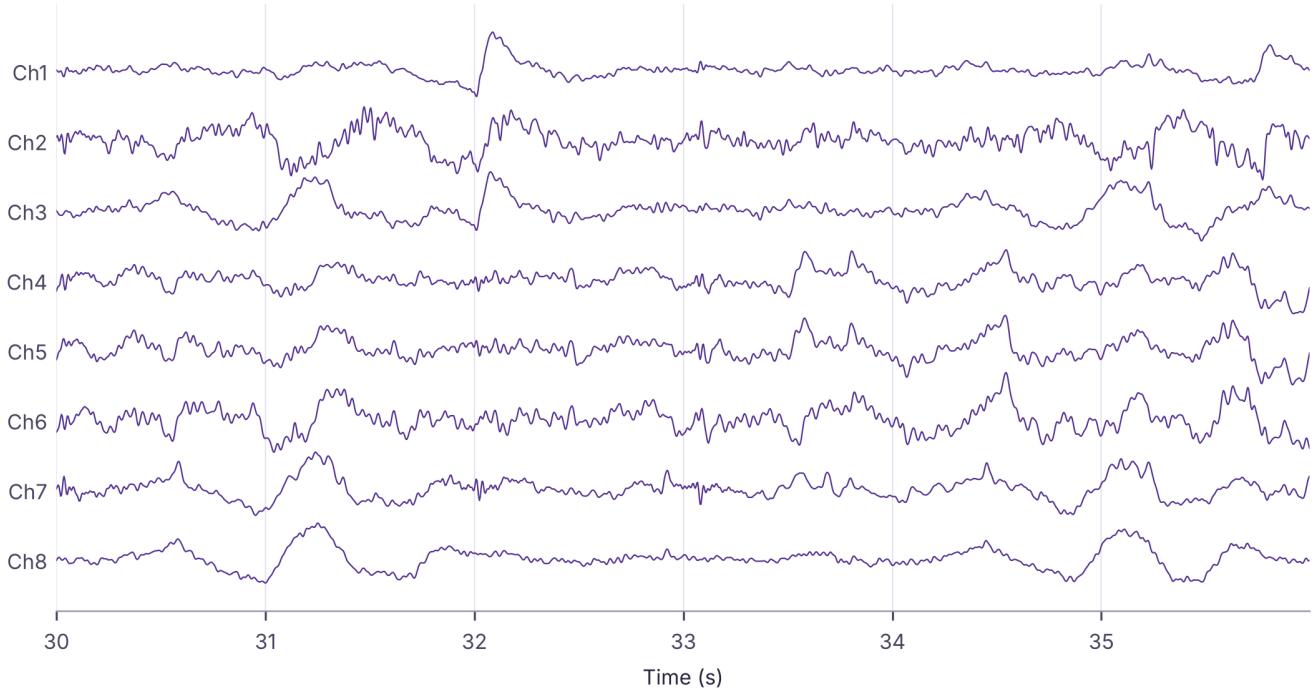


Figure 2 — 6-second excerpt of Datasets/dataset.xlsx, 50 Hz notch + 1–40 Hz band-pass; each trace auto-scaled.

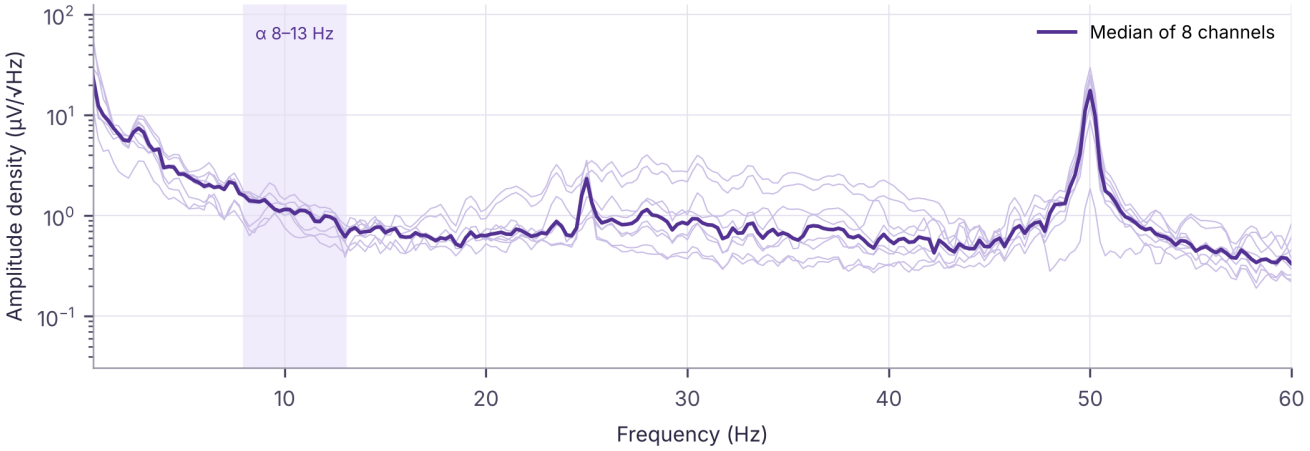


Figure 3 — Amplitude spectral density of the raw recording (Welch, 4 s segments); bold line = median of 8 channels.

09 RELATED DOCUMENTS

PE8-DS-001 Product Datasheet	**PE8-MD-002** Mechanical & Dimensional Drawing	**PE8-PCB-003** PCB & Manufacturing Specification
PE8-PIN-004 Connector Pinout & Electrode Guide	**PE8-QSG-005** Quick Start & Safety Guide	**PE8-ENC-007** 3D-Printed Enclosure Guide